# ASSET MANAGEMENT SYSTEM

## API Documentation

Version 2.0

Last Updated: January 8, 2026

**Jirani Smart Limited**

# Asset Management System - API Documentation

Version: 2.0

Last Updated: January 8, 2026

Base URL: `/api`

# Quick Reference - All Endpoints

## Authentication Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/auth/register` | Register new user | Admin |
| POST | `/api/auth/login` | User login | No |
| POST | `/api/auth/logout` | User logout | Yes |
| POST | `/api/auth/refresh-token` | Refresh access token | Refresh Token |
| GET | `/api/auth/roles` | Get all user roles | Yes |
| PUT | `/api/auth/update-user/:id` | Update user | Yes |

## Asset Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/assets/` | Get all assets | Admin, Standard User |
| GET | `/api/assets/:id` | Get asset by ID | Admin, Standard User |
| GET | `/api/assets/search` | Search assets | Admin, Standard User |
| GET | `/api/assets/statuses/list` | Get status list | Admin, Standard User |
| POST | `/api/assets/` | Create asset | Admin |
| PUT | `/api/assets/:id` | Update asset | Admin |
| DELETE | `/api/assets/:id` | Delete asset | Admin |

## Asset Type Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/asset-types/create` | Create asset type | Admin |

| | | | |
|---|---|---|---|
| GET | `/api/asset-types` | Get all asset types | Yes |
| GET | `/api/asset-types/:id` | Get asset type by ID | Yes |
| PUT | `/api/asset-types/:id` | Update asset type | Admin |
| DELETE | `/api/asset-types/:id` | Delete asset type | Admin |
| POST | `/api/assets/asset-types/create` | Create asset type (alt) | Admin |
| GET | `/api/assets/asset-types/all` | Get all asset types (alt) | Yes |

## Asset Status Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/assets/asset-statuses/create` | Create status | Admin, Super Admin |
| GET | `/api/assets/asset-statuses/all` | Get all statuses | Yes |
| GET | `/api/assets/asset-statuses/:id` | Get status by ID | Yes |
| PUT | `/api/assets/asset-statuses/:id` | Update status | Admin, Super Admin |
| DELETE | `/api/assets/asset-statuses/:id` | Delete status | Admin, Super Admin |

## Asset Attachment Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/asset-attachments/:assetId/attachments` | Upload attachment | Admin, Standard User |
| GET | `/api/asset-attachments/:assetId/attachments` | Get attachments | Admin, Standard User |
| DELETE | `/api/asset-attachments/attachments/:id` | Delete attachment | Admin |

## Assignment Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/assignments/` | Assign asset | Admin |
| GET | `/api/assignments/:id` | Get assignment by ID | Admin, Standard User |
| PUT | `/api/assignments/:id/return` | Return asset | Admin |
| GET | `/api/assignments/asset/:assetId/history` | Get asset history | Yes |

## Assignment Attachment Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/assignment-attachments/:assignmentId/attachments` | Upload attachment | Admin, Standard User |
| GET | `/api/assignment-attachments/:assignmentId/attachments` | Get attachments | Admin, Standard User |
| DELETE | `/api/assignment-attachments/attachments/:id` | Delete attachment | Admin |

## Employee Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/employees/:id` | Get employee by ID | Yes |
| GET | `/api/employees/:employeeId/assets` | Get employee assets | Yes |

## Expense Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|

| | | | |
|---|---|---|---|
| GET | `/api/expenses/:id` | Get expense by ID | Yes |
| POST | `/api/expenses/` | Add expense | Admin |

## Expense Type Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/expenses/expense-types/create` | Create expense type | Admin |
| GET | `/api/expenses/expense-types/all` | Get all expense types | Yes |
| GET | `/api/expenses/expense-types/:id` | Get expense type by ID | Yes |
| PUT | `/api/expenses/expense-types/:id` | Update expense type | Admin |
| DELETE | `/api/expenses/expense-types/:id` | Delete expense type | Admin |

## Expense Attachment Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/expense-attachments/:expenseId/attachments` | Upload attachment | Admin, Standard User |
| GET | `/api/expense-attachments/:expenseId/attachments` | Get attachments | Admin, Standard User |
| DELETE | `/api/expense-attachments/attachments/:id` | Delete attachment | Admin |

## Department Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/departments/` | Get all departments | No |

| | | | |
|---|---|---|---|
| GET | `/api/departments/:id` | Get department by ID | No |
| POST | `/api/departments/` | Create department | Admin |
| PUT | `/api/departments/:id` | Update department | Admin |
| DELETE | `/api/departments/:id` | Delete department | Admin |

## Branch Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/branches/` | Get all branches | No |
| GET | `/api/branches/:id` | Get branch by ID | No |
| POST | `/api/branches/` | Create branch | No |
| PUT | `/api/branches/:id` | Update branch | No |
| DELETE | `/api/branches/:id` | Delete branch | No |

## User Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/users/:id` | Get user by ID | Yes |
| PUT | `/api/users/profile` | Update profile | Yes |
| POST | `/api/users/change-password` | Change password | Yes |
| POST | `/api/users/reset-password/:id` | Reset password | Yes |
| POST | `/api/users/toggle-status/:id` | Toggle user status | Yes |

## Report Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|

| | | | |
|---|---|---|---|
| GET | `/api/reports/assets` | Get filtered assets report | Yes |
| GET | `/api/reports/assets/all` | Get all assets report | Yes |
| GET | `/api/reports/assets/assignments` | Get assets assignments | Yes |
| GET | `/api/reports/assets/export` | Export assets to Excel | Yes |
| GET | `/api/reports/assets/employee/:employeeId` | Get assets by employee | Yes |
| GET | `/api/reports/assets/branch/:location` | Get assets by branch | Yes |
| GET | `/api/reports/expenses` | Get expenses report | Yes |
| GET | `/api/reports/expenses/all` | Get all expenses | Yes |
| GET | `/api/reports/expenses/export` | Export expenses to Excel | Yes |
| GET | `/api/reports/expenses/time-period` | Get expenses by period | Admin |
| GET | `/api/reports/assignments` | Get assignments report | Yes |
| GET | `/api/reports/assignments/export` | Export assignments to Excel | Yes |
| GET | `/api/reports/action-logs` | Get action logs report | Yes |
| GET | `/api/reports/action-logs/export` | Export action logs to Excel | Yes |

## Bulk Upload Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | `/api/upload/assets` | Bulk upload assets | Admin |

### System Configuration Endpoints

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | `/api/system-config/` | Get system config | Admin |
| GET | `/api/system-config/public` | Get public config | No |
| PUT | `/api/system-config/` | Update system config | Admin |
| POST | `/api/system-config/upload-logo` | Upload company logo | Admin |

## Table of Contents

# Authentication

## Register User

Endpoint: `POST /api/auth/register`

Authorization: Required (Admin)

Description: Register a new user in the system

Request Body:

```
{
  "first_name": "Caleb",
  "middle_name": "John",
  "last_name": "Kiprono",
  "email": "user@jiranismart.com",
  "password": "securePassword123",
  "phone": "+254712345678",
  "role_id": 1,
  "department_id": 2,
  "department": "IT",
  "branch_id": 1,
  "branch_location": "Head Office"
}
```

Success Response:

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "id": 1,
    "employee_id": 1,
    "first_name": "Caleb",
    "middle_name": "John",
    "last_name": "Kiprono",
    "email": "user@jiranismart.com",
    "phone": "+254712345678"
  }
}
```

Note: This endpoint creates both an employee record and a user account in a single transaction.

---

## Login

Endpoint: `POST /api/auth/login`

Authorization: None

Description: Authenticate user and receive access tokens

Request Body:

```
{
  "email": "user@jiranismart.com",
  "password": "securePassword123"
}
```

Success Response:

```
{
  "success": true,
  "message": "Logged in successfully",
  "data": {
    "user": {
      "id": 1,
      "employee_id": 1,
      "first_name": "Dev",
      "middle_name": "Dev",
      "last_name": "Dev",
      "email": "ict@jiranismart.com",
      "password": "$2a$10$pvN0OKHi7HAfYAoid6F3.e9ag3r57KVmPL/tezhi6zubmPPAlFp.C",
      "role_id": 1,
      "department_id": null,
      "phone": "+254793577021",
      "branch_id": 1,
      "is_active": true,
      "role": "Admin"
    },
    "accessToken":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJpY3RAamlyYW5pc21hcnQuY
29tIiwicm9sZSI6IkFkbWluIiwiaWF0IjoxNzY3ODcxODI2LCJleHAiOjE3Njc4NzU0MjZ9.QP31Z7CYXWp
_9DToqoqelYF9zcgU75N1CF51VLzxe6Y",
    "refreshToken":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJpY3RAamlyYW5pc21hcnQuY
29tIiwicm9sZSI6IkFkbWluIiwiaWF0IjoxNzY3ODcxODI2LCJleHAiOjE3Njg0NzY2MjZ9.e_EcvjH1pQh
Dk6DI-IGYrBm5AiWb-HgfFsRRJOgUCVY",
    "returnTo": "/dashboard"
  }
}
```

Note: The `password` field in the response contains the hashed password. For security, this should not be exposed in production environments.

Cookies Set:

- `accessToken` - HTTP-only, 30 minutes expiry
- `refreshToken` - HTTP-only, 7 days expiry

### Refresh Token

Endpoint: `POST /api/auth/refresh-token`

Authorization: Refresh token in cookies

Description: Refresh the access token using refresh token

Success Response:

```
{
  "success": true,
  "message": "Token refreshed successfully"
}
```

---

### Logout

Endpoint: `POST /api/auth/logout`

Authorization: Required

Description: Logout user and clear authentication cookies

Success Response:

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

---

### Get All User Roles

Endpoint: `GET /api/auth/roles`

Authorization: Required

Description: Retrieve all available user roles

Success Response:

```
{
  "success": true,
```

```
  "message": "Roles retrieved successfully",
  "data": [
    {
      "id": 1,
      "role_name": "Admin"
    },
    {
      "id": 2,
      "role_name": "Standard User"
    }
  ]
}
```

---

## Update User

Endpoint: `PUT /api/auth/update-user/:id`

Authorization: Required

Description: Update user information

URL Parameters:

- `id` - User ID

Request Body:

```
{
  "email": "newemail@jiranismart.com",
  "first_name": "Jane",
  "last_name": "Smith",
  "role_id": 2
}
```

---

# Assets

## Get All Assets

Endpoint: `GET /api/assets/`

Authorization: Required (Admin, Standard User)

Description: Retrieve all assets in the system

Success Response:


```
{
  "success": true,
  "message": "Assets retrieved successfully",
  "data": {
    "assets": [
      {
        "id": 1,
        "asset_tag": "ASSET-001",
        "asset_type": "Laptop",
        "manufacturer": "Dell",
        "model": "Latitude 5420",
        "serial_number": "SN123456",
        "status": "In Use",
        "purchase_date": "2025-01-01",
        "purchase_price": 1200,
        "notes": "Assigned to IT department",
        "type_name": "Computer Equipment",
        "status_name": "In Use",
        "location": "Head Office"
      }
    ],
    "total": 150,
    "page": 1,
    "itemsPerPage": 20,
    "totalPages": 8
  }
}
```


---

## Get Asset By ID

Endpoint: `GET /api/assets/:id`

Authorization: Required (Admin, Standard User)

Description: Retrieve a specific asset by ID

URL Parameters:

- `id` - Asset ID

Success Response:


```
{
  "success": true,
  "message": "Asset retrieved successfully",
  "data": {
    "id": 1,
    "asset_tag": "ASSET-001",
    "asset_type": "Laptop",
    "manufacturer": "Dell",
    "model": "Latitude 5420",
    "serial_number": "SN12345678",
    "status": "In Use",
    "purchase_date": "2025-01-01",
    "purchase_price": 1200,
    "notes": "Assigned to IT department",
    "asset_type_id": 1,
    "asset_status_id": 1,
    "branch_id": 1,
    "type_name": "Computer Equipment",
    "status_name": "In Use",
    "branch_name": "Head Office",
    "location": "New York"
  }
}
```


---

## Search Assets

Endpoint: `GET /api/assets/search`

Authorization: Required (Admin, Standard User)

Description: Search assets with query parameters

Query Parameters:

- `asset_tag` - Asset tag (partial match)
- `asset_name` - Asset name (partial match)
- `status_id` - Status ID
- `asset_type_id` - Asset type ID
- `branch_id` - Branch ID
- `department_id` - Department ID

Example: `GET /api/assets/search?asset_name=laptop&status_id=1`

---

### Get Asset Statuses List

Endpoint: `GET /api/assets/statuses/list`

Authorization: Required (Admin, Standard User)

Description: Get list of all asset statuses

Success Response:

```
{
  "success": true,
  "message": "Assets statuses retrieved successfully",
  "data": [
    {
      "id": 1,
      "status_name": "Available",
      "description": "Asset is available for assignment"
    },
    {
      "id": 2,
      "status_name": "In Use",
      "description": "Asset is currently assigned"
    }
  ]
}
```

---

### Create Asset

Endpoint: `POST /api/assets/`

Authorization: Required (Admin)

Description: Create a new asset

Request Body:

```
{
  "asset_tag": "ASSET-002",
  "asset_type": "Laptop",
  "manufacturer": "HP",
  "model": "EliteBook 840",
  "serial_number": "HP98765432",
  "status": "Available",
  "purchase_date": "2025-01-05",
  "purchase_price": 1500,
  "notes": "New HP laptop for development team",
  "asset_type_id": 1,
  "asset_status_id": 1,
  "branch_id": 1
}
```

Success Response:


```
{
  "success": true,
  "message": "Asset created successfully",
  "data": {
    "id": 2,
    "asset_tag": "ASSET-002",
    "asset_type": "Laptop",
    "manufacturer": "HP",
    "model": "EliteBook 840",
    "serial_number": "HP98765432",
    "status": "Available",
    "purchase_date": "2025-01-05",
    "purchase_price": 1500,
    "notes": "New HP laptop for development team",
    "asset_type_id": 1,
    "asset_status_id": 1,
    "branch_id": 1
  }
}
```


---

## Update Asset

Endpoint: `PUT /api/assets/:id`

Authorization: Required (Admin)

Description: Update an existing asset

URL Parameters:

- `id` - Asset ID

Request Body: Same as Create Asset

---

## Delete Asset

Endpoint: `DELETE /api/assets/:id`

Authorization: Required (Admin)

Description: Delete an asset

URL Parameters:

- `id` - Asset ID

Success Response:

```
{
  "success": true,
  "message": "Asset deleted successfully"
}
```

# Asset Types

## Create Asset Type

Endpoint: `POST /api/asset-types/create` or `POST /api/assets/asset-types/create`

Authorization: Required (Admin)

Description: Create a new asset type

Request Body:

```
{
  "name": "Office Furniture",
  "description": "Desks, chairs, and cabinets"
}
```

Success Response:

```
{
  "success": true,
  "message": "Asset type created successfully",
  "data": {
    "id": 5,
    "name": "Office Furniture",
    "description": "Desks, chairs, and cabinets",
    "created_at": "2025-01-05T10:00:00Z",
    "updated_at": "2025-01-05T10:00:00Z"
  }
}
```

---

## Get All Asset Types

Endpoint: `GET /api/asset-types` or `GET /api/assets/asset-types/all`

Authorization: Required

Description: Retrieve all asset types

Success Response:

```
{
  "success": true,
  "message": "Asset types retrieved successfully",
  "data": [
    {
      "id": 1,
      "name": "Computer Equipment",
      "description": "Laptops, desktops, monitors",
```

```
      "created_at": "2025-01-01T10:00:00Z",
      "updated_at": "2025-01-01T10:00:00Z"
    }
  ]
}
```

---

### Get Asset Type By ID

Endpoint: `GET /api/asset-types/:id`

Authorization: Required

Description: Retrieve a specific asset type with its details

URL Parameters:

- `id` - Asset type ID

Success Response:

```
{
  "success": true,
  "message": "Asset type retrieved successfully",
  "data": {
    "id": 1,
    "name": "Computer Equipment",
    "description": "Laptops, desktops, monitors, and peripherals",
    "created_at": "2025-01-01T10:00:00Z",
    "updated_at": "2025-01-01T10:00:00Z"
  }
}
```

---

### Update Asset Type

Endpoint: `PUT /api/asset-types/:id`

Authorization: Required (Admin)

Description: Update an asset type

URL Parameters:

- `id` - Asset type ID

Request Body:

```
{
  "name": "Updated Type Name",
  "description": "Updated description"
}
```

---

## Delete Asset Type

Endpoint: `DELETE /api/asset-types/:id`

Authorization: Required (Admin)

Description: Delete an asset type

URL Parameters:

- `id` - Asset type ID

---

# Asset Statuses

## Create Asset Status

Endpoint: `POST /api/assets/asset-statuses/create`

Authorization: Required (Admin, Super Admin)

Description: Create a new asset status

Request Body:

```
{
  "status_name": "Under Repair",
  "description": "Asset is being repaired"
}
```

---

## Get All Asset Statuses

Endpoint: `GET /api/assets/asset-statuses/all`

Authorization: Required

Description: Retrieve all asset statuses

---

## Get Asset Status By ID

Endpoint: `GET /api/assets/asset-statuses/:id`

Authorization: Required

Description: Retrieve a specific asset status with its details

URL Parameters:

- `id` - Asset status ID

Success Response:

```
{
  "success": true,
  "message": "Asset status retrieved successfully",
```

```
  "data": {
    "id": 1,
    "status_name": "Available",
    "description": "Asset is available for assignment",
    "color_code": "#28a745",
    "is_active": true,
    "created_at": "2025-01-01T10:00:00Z"
  }
}
```


---

### Update Asset Status

Endpoint: `PUT /api/assets/asset-statuses/:id`

Authorization: Required (Admin, Super Admin)

Description: Update an asset status

URL Parameters:

- `id` - Asset status ID

---

### Delete Asset Status

Endpoint: `DELETE /api/assets/asset-statuses/:id`

Authorization: Required (Admin, Super Admin)

Description: Delete an asset status

URL Parameters:

- `id` - Asset status ID

---

# Asset Attachments

## Upload Asset Attachment

Endpoint: `POST /api/asset-attachments/:assetId/attachments`

Authorization: Required (Admin, Standard User)

Description: Upload an attachment for an asset

Content-Type: `multipart/form-data`

URL Parameters:

- `assetId` - Asset ID

Form Data:

- `file` - File to upload (max 10MB)

Supported File Types:

- Documents: PDF, DOC, DOCX, XLS, XLSX
- Images: JPG, JPEG, PNG, GIF, BMP
- Archives: ZIP, RAR
- Text: TXT, CSV

Success Response:

```
{
  "success": true,
  "message": "Attachment uploaded successfully",
  "data": {
    "id": 1,
    "asset_id": 1,
    "file_name": "invoice.pdf",
    "file_path": "/uploads/assets/invoice.pdf",
    "file_size": 102400,
    "mime_type": "application/pdf",
    "uploaded_by": 5,
    "uploaded_at": "2026-01-08T10:30:00Z"
  }
}
```

Error Response (File Too Large):

```
{
  "success": false,
  "message": "File size exceeds maximum limit of 10MB"
}
```

## Get Asset Attachments

Endpoint: `GET /api/asset-attachments/:assetId/attachments`

Authorization: Required (Admin, Standard User)

Description: Get all attachments for an asset

URL Parameters:

- `assetId` - Asset ID

Success Response:

```
{
  "success": true,
  "message": "Attachments retrieved successfully",
  "data": [
    {
      "id": 1,
      "asset_id": 1,
      "file_name": "invoice.pdf",
      "file_path": "/uploads/assets/invoice.pdf",
      "file_size": 102400,
      "uploaded_at": "2026-01-08T10:30:00Z"
    }
  ]
}
```

## Delete Asset Attachment

Endpoint: `DELETE /api/asset-attachments/attachments/:id`

Authorization: Required (Admin)

Description: Delete an asset attachment

URL Parameters:

- `id` - Attachment ID

Success Response:

```
{
  "success": true,
  "message": "Attachment deleted successfully"
```

```
}
```

Error Response (Not Found):

```
{
  "success": false,
  "message": "Attachment not found"
}
```

# Assignments

## Assign Asset

Endpoint: `POST /api/assignments/`

Authorization: Required (Admin)

Description: Assign an asset to an employee

Request Body:

```
{
  "asset_id": 1,
  "employee_id": 5,
  "assigned_date": "2026-01-08",
  "notes": "Laptop for development work"
}
```

Success Response:

```
{
  "success": true,
  "message": "Asset assigned successfully",
  "data": {
    "id": 10,
    "asset_id": 1,
    "employee_id": 5,
    "assigned_date": "2026-01-08",
    "return_date": null,
    "notes": "Laptop for development work"
  }
}
```

---

## Get Assignment By ID

Endpoint: `GET /api/assignments/:id`

Authorization: Required (Admin, Standard User)

Description: Retrieve assignment details

URL Parameters:

- `id` - Assignment ID

---

### Return Asset

Endpoint: `PUT /api/assignments/:id/return`

Authorization: Required (Admin)

Description: Mark an asset as returned

URL Parameters:

- `id` - Assignment ID

Request Body:

```
{
  "return_date": "2026-01-08",
  "return_notes": "Asset returned in good condition"
}
```

Success Response:

```
{
  "success": true,
  "message": "Asset returned successfully",
  "data": {
    "id": 10,
    "asset_id": 1,
    "employee_id": 5,
    "assigned_date": "2026-01-01",
    "return_date": "2026-01-08",
    "return_notes": "Asset returned in good condition"
  }
}
```

---

### Get Asset Assignment History

Endpoint: `GET /api/assignments/asset/:assetId/history`

Authorization: Required

Description: Get assignment history for a specific asset

URL Parameters:

- `assetId` - Asset ID

Success Response:

```
{
  "success": true,
  "message": "Asset history retrieved successfully",
  "data": [
    {
      "id": 10,
      "asset_id": 1,
      "asset_tag": "ASSET-001",
      "employee_id": 5,
      "first_name": "Caleb",
      "middle_name": null,
      "last_name": "Kiprono",
      "assigned_date": "2026-01-01",
      "return_date": "2026-01-08",
      "notes": "Laptop for development work"
    }
  ]
}
```

# Assignment Attachments

## Upload Assignment Attachment

Endpoint: `POST /api/assignment-attachments/:assignmentId/attachments`

Authorization: Required (Admin, Standard User)

Description: Upload an attachment for an assignment

Content-Type: `multipart/form-data`

URL Parameters:

- `assignmentId` - Assignment ID

Form Data:

- `file` - File to upload (max 10MB)

Success Response:

```
{
  "success": true,
  "message": "Attachment uploaded successfully",
  "data": {
    "id": 5,
    "assignment_id": 10,
    "file_name": "handover_form.pdf",
    "file_path": "/uploads/assignments/handover_form.pdf",
    "file_size": 85000,
    "mime_type": "application/pdf",
    "uploaded_by": 5,
    "uploaded_at": "2026-01-08T10:30:00Z"
  }
}
```

---

## Get Assignment Attachments

Endpoint: `GET /api/assignment-attachments/:assignmentId/attachments`

Authorization: Required (Admin, Standard User)

Description: Get all attachments for an assignment

URL Parameters:

- `assignmentId` - Assignment ID

Success Response:


```
{
  "success": true,
  "message": "Attachments retrieved successfully",
  "data": [
    {
      "id": 5,
      "assignment_id": 10,
      "file_name": "handover_form.pdf",
      "file_path": "/uploads/assignments/handover_form.pdf",
      "file_size": 85000,
      "mime_type": "application/pdf",
      "uploaded_by": 5,
      "uploaded_at": "2026-01-08T10:30:00Z"
    }
  ]
}
```


---

## Delete Assignment Attachment

Endpoint: `DELETE /api/assignment-attachments/attachments/:id`

Authorization: Required (Admin)

Description: Delete an assignment attachment

URL Parameters:

- `id` - Attachment ID

Success Response:


```
{
  "success": true,
  "message": "Attachment deleted successfully"
}
```


---

# Employees

## Get Employee By ID

Endpoint: `GET /api/employees/:id`

Authorization: Required

Description: Retrieve employee details including personal information and employment data

URL Parameters:

- `id` - Employee ID

Success Response:

```
{
  "success": true,
  "message": "Employee retrieved successfully",
  "data": {
    "id": 5,
    "first_name": "Caleb",
    "middle_name": null,
    "last_name": "Kiprono",
    "department_id": 1,
    "department": "IT",
    "branch_id": 1,
    "branch_location": "Head Office",
    "hire_date": "2024-06-01",
    "status": "Active"
  }
}
```

Error Response:

```
{
  "success": false,
  "message": "Employee not found"
}
```

---

## Get Employee Assets

Endpoint: `GET /api/employees/:employeeId/assets`

Authorization: Required

Description: Get all assets assigned to an employee

URL Parameters:

- `employeeId` - Employee ID

Success Response:

```
{
  "success": true,
  "message": "Employee assets retrieved successfully",
  "data": [
    {
      "asset_id": 1,
      "asset_tag": "ASSET-001",
      "asset_name": "Dell Laptop",
      "assigned_date": "2026-01-01",
      "assignment_id": 10
    }
  ]
}
```

# Expenses

## Get Expense By ID

Endpoint: `GET /api/expenses/:id`

Authorization: Required

Description: Retrieve expense details

URL Parameters:

- `id` - Expense ID

---

## Add Expense

Endpoint: `POST /api/expenses/`

Authorization: Required (Admin)

Description: Record a new expense

Request Body:

```
{
  "asset_id": 1,
  "expense_type_id": 2,
  "expense_date": "2026-01-08",
  "amount": 250,
  "description": "Screen replacement",
  "vendor": "Tech Repairs Inc"
}
```

Success Response:

```
{
  "success": true,
  "message": "Expense added successfully",
  "data": {
    "id": 15,
    "asset_id": 1,
    "expense_type_id": 2,
    "expense_date": "2026-01-08",
    "amount": 250,
    "description": "Screen replacement",
    "vendor": "Tech Repairs Inc"
  }
}
```

# Expense Types

## Create Expense Type

Endpoint: `POST /api/expenses/expense-types/create`

Authorization: Required (Admin)

Description: Create a new expense type

Request Body:

```
{
  "name": "Upgrade",
  "description": "Hardware or software upgrades"
}
```

---

## Get All Expense Types

Endpoint: `GET /api/expenses/expense-types/all`

Authorization: Required

Description: Retrieve all expense types

---

## Get Expense Type By ID

Endpoint: `GET /api/expenses/expense-types/:id`

Authorization: Required

Description: Retrieve a specific expense type with its details

URL Parameters:

- `id` - Expense type ID

Success Response:

```
{
  "success": true,
  "message": "Expense type retrieved successfully",
```

```
  "data": {
    "id": 2,
    "name": "Repair",
    "description": "Maintenance and repair costs",
    "created_at": "2025-01-01T10:00:00Z",
    "updated_at": "2025-01-01T10:00:00Z"
  }
}
```

---

## Update Expense Type

Endpoint: `PUT /api/expenses/expense-types/:id`

Authorization: Required (Admin)

Description: Update an expense type

URL Parameters:

- `id` - Expense type ID

---

## Delete Expense Type

Endpoint: `DELETE /api/expenses/expense-types/:id`

Authorization: Required (Admin)

Description: Delete an expense type

URL Parameters:

- `id` - Expense type ID

---

# Expense Attachments

## Upload Expense Attachment

Endpoint: `POST /api/expense-attachments/:expenseId/attachments`

Authorization: Required (Admin, Standard User)

Description: Upload an attachment for an expense (e.g., receipt, invoice)

Content-Type: `multipart/form-data`

URL Parameters:

- `expenseId` - Expense ID

Form Data:

- `file` - File to upload (max 10MB)

Success Response:

```
{
  "success": true,
  "message": "Attachment uploaded successfully",
  "data": {
    "id": 8,
    "expense_id": 15,
    "file_name": "repair_receipt.jpg",
    "file_path": "/uploads/expenses/repair_receipt.jpg",
    "file_size": 245000,
    "mime_type": "image/jpeg",
    "uploaded_by": 5,
    "uploaded_at": "2026-01-08T10:30:00Z"
  }
}
```

---

## Get Expense Attachments

Endpoint: `GET /api/expense-attachments/:expenseId/attachments`

Authorization: Required (Admin, Standard User)

Description: Get all attachments for an expense

URL Parameters:

- `expenseId` - Expense ID

Success Response:


```
{
  "success": true,
  "message": "Attachments retrieved successfully",
  "data": [
    {
      "id": 8,
      "expense_id": 15,
      "file_name": "repair_receipt.jpg",
      "file_path": "/uploads/expenses/repair_receipt.jpg",
      "file_size": 245000,
      "mime_type": "image/jpeg",
      "uploaded_by": 5,
      "uploaded_at": "2026-01-08T10:30:00Z"
    }
  ]
}
```


---

## Delete Expense Attachment

Endpoint: `DELETE /api/expense-attachments/attachments/:id`

Authorization: Required (Admin)

Description: Delete an expense attachment

URL Parameters:

- `id` - Attachment ID

Success Response:


```
{
  "success": true,
  "message": "Attachment deleted successfully"
}
```


---

# Departments

## Get All Departments

Endpoint: `GET /api/departments/`

Authorization: None

Description: Retrieve all departments

Success Response:

```
{
  "success": true,
  "message": "Departments retrieved successfully",
  "data": [
    {
      "id": 1,
      "name": "IT",
      "created_at": "2025-01-01T10:00:00Z",
      "updated_at": "2025-01-01T10:00:00Z"
    }
  ]
}
```

---

## Get Department By ID

Endpoint: `GET /api/departments/:id`

Authorization: None

Description: Retrieve a specific department with its details

URL Parameters:

- `id` - Department ID

Success Response:

```
{
  "success": true,
  "message": "Department retrieved successfully",
  "data": {
    "id": 1,
    "name": "IT",
    "created_at": "2025-01-01T10:00:00Z",
    "updated_at": "2025-01-01T10:00:00Z"
  }
}
```

## Create Department

Endpoint: `POST /api/departments/`

Authorization: Required (Admin)

Description: Create a new department

Request Body:

```
{
  "name": "Marketing"
}
```

## Update Department

Endpoint: `PUT /api/departments/:id`

Authorization: Required (Admin)

Description: Update a department's information

URL Parameters:

- `id` - Department ID

Request Body:

```
{
  "department_name": "Information Technology",
  "name": "Information Technology"
}
```

Success Response:

```
{
  "success": true,
  "message": "Department updated successfully",
  "data": {
    "id": 1,
    "name": "Information Technology",
    "created_at": "2025-01-01T10:00:00Z",
    "updated_at": "2025-01-15T14:30:00Z
```

```
}
```

---

### Delete Department

Endpoint: `DELETE /api/departments/:id`

Authorization: Required (Admin)

Description: Delete a department (only if no employees are assigned)

URL Parameters:

- `id` - Department ID

Success Response:

```
{
  "success": true,
  "message": "Department deleted successfully"
}
```

Error Response (Has Dependencies):

```
{
  "success": false,
  "message": "Cannot delete department. It has assigned employees."
}
```

---

# Branches

## Get All Branches

Endpoint: `GET /api/branches/`

Authorization: None

Description: Retrieve all branches

Success Response:

```
{
  "success": true,
  "message": "Branches retrieved successfully",
  "data": [
    {
      "id": 1,
      "name": "Head Office",
      "location": "New York",
      "created_at": "2025-01-15T10:30:00.000Z",
      "updated_at": "2025-01-15T10:30:00.000Z"
    }
  ]
}
```

---

## Get Branch By ID

Endpoint: `GET /api/branches/:id`

Authorization: None

Description: Retrieve a specific branch

URL Parameters:

- `id` - Branch ID

---

## Create Branch

Endpoint: `POST /api/branches/`

Authorization: None

Description: Create a new branch

Request Body:

```
{name": "West Coast Office",
  "location": "San Francisco
  "address": "456 Market Street"
}
```

---

## Update Branch

Endpoint: `PUT /api/branches/:id`

Authorization: None

Description: Update a branch's information

URL Parameters:

- `id` - Branch ID

Request Body:

```
{
  "branch_name": "West Coast Office - Updated",
  "name": "West Coast Office - Updated",
  "location": "San Francisco, CA"
}
```

Success Response:

```
{
  "success": true,
  "message": "Branch updated successfully",
  "data": {
    "id": 2,
    "name": "West Coast Office - Updated",
    "location": "San Francisco, CA",
    "created_at": "2025-01-15T10:30:00.000Z",
    "updated_at": "2025-01-20T14:45:00.000Z
}
```

---

### Delete Branch

Endpoint: `DELETE /api/branches/:id`

Authorization: None

Description: Delete a branch (only if no assets or employees are assigned)

URL Parameters:

- `id` - Branch ID

Success Response:

```
{
  "success": true,
  "message": "Branch deleted successfully"
}
```

Error Response (Has Dependencies):

```
{
  "success": false,
  "message": "Cannot delete branch. It has assigned assets or employees."
}
```

# Users

## Get User By ID

Endpoint: `GET /api/users/:id`

Authorization: Required

Description: Retrieve user details including profile and role information

URL Parameters:

- `id` - User ID

Success Response:


```
{
  "success": true,
  "message": "User retrieved successfully",
  "data": {
    "id": 1,
    "employee_id": 1,
    "first_name": "Dev",
    "middle_name": "Dev",
    "last_name": "Dev",
    "email": "ict@jiranismart.com",
    "password": "$2a$10$pvN0OKHi7HAfYAoid6F3.e9ag3r57KVmPL/tezhi6zubmPPAlFp.C",
    "role_id": 1,
    "department_id": null,
    "phone": "+254793577021",
    "branch_id": 1,
    "is_active": true,
    "name": "Head Office",
    "location": "New York",
    "role_name": "Admin",
    "department_name": null,
    "employee_first_name": "Dev",
    "employee_middle_name": "Dev",
    "employee_last_name": "Dev",
    "departmnt_id": null
  }
}
```


Note: Response includes joined data from branches, roles, employees, and departments tables.

---

## Update User Profile

Endpoint: `PUT /api/users/profile`

Authorization: Required

Description: Update current user's profile information

Request Body:

```
{
  "first_name": "Jane",
  "last_name": "Smith",
  "email": "jane.smith@jiranismart.com",
  "phone": "+254723456789"
}
```

Success Response:

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": 5,
    "email": "jane.smith@jiranismart.com",
    "first_name": "Jane",
    "last_name": "Smith",
    "phone": "+254723456789"
  }
}
```

Error Response (Email Already Exists):

```
{
  "success": false,
  "message": "Email already in use"
}
```

---

## Change Password

Endpoint: `POST /api/users/change-password`

Authorization: Required

Description: Change current user's password

Request Body:

```
{
  "current_password": "oldPassword123",
  "new_password": "newSecurePassword456",
  "confirm_password": "newSecurePassword456"
}
```

Success Response:

```
{
  "success": true,
  "message": "Password changed successfully"
}
```

---

## Reset Password

Endpoint: `POST /api/users/reset-password/:id`

Authorization: Required (Admin)

Description: Reset a user's password (Admin function - used to set temporary password)

URL Parameters:

- `id` - User ID

Request Body:

```
{
  "password": "TempPassword@123"
}
```

Success Response:

```
{
  "success": true,
  "message": "Password reset successfully"
}
```

Note: User should be prompted to change this temporary password on next login.

---

## Toggle User Status

Endpoint: `POST /api/users/toggle-status/:id`

Authorization: Required (Admin)

Description: Activate or deactivate a user account (prevents login when deactivated)

URL Parameters:

- `id` - User ID

Request Body:

```
{
  "is_active": false
}
```

Success Response:

```
{
  "success": true,
  "message": "User status updated successfully",
  "data": {
    "id": 8,
    "email": "user@jiranismart.com",
    "is_active": false
  }
}
```

Note: Deactivated users cannot log in but their data is preserved in the system.

# Reports

## Get All Assets Report

Endpoint: `GET /api/reports/assets/all`

Authorization: Required

Description: Get all assets without filters

Success Response:

```
{
  "success": true,
  "message": "Assets report generated successfully",
  "data": [
    {
      "id": 1,
      "asset_tag": "ASSET-001",
      "asset_name": "Dell Laptop",
      "asset_type": "Computer Equipment",
      "status": "In Use",
      "branch": "Head Office",
      "department": "IT",
      "purchase_price": 1200,
      "current_value": 1000
    }
  ]
}
```

---

## Get Filtered Assets Report

Endpoint: `GET /api/reports/assets`

Authorization: Required

Description: Get filtered assets report with pagination

Query Parameters:

- `asset_tag` - Filter by asset tag (partial match)
- `asset_name` - Filter by asset name (partial match)
- `status_id` - Filter by status ID
- `asset_type_id` - Filter by asset type ID
- `branch_id` - Filter by branch ID
- `department_id` - Filter by department ID
- `limit` - Number of records per page (default: 20)

- `offset` - Starting record number (default: 0)

Example: `GET /api/reports/assets?status_id=1&limit=50&offset=0`

Success Response:


```
{
  "success": true,
  "message": "Filtered assets report generated successfully",
  "data": {
    "items": [
      {
        "id": 1,
        "asset_tag": "ASSET-001",
        "asset_name": "Dell Laptop",
        "asset_type": "Computer Equipment",
        "status": "In Use",
        "branch": "Head Office",
        "department": "IT"
      }
    ],
    "total": 150,
    "limit": 50,
    "offset": 0
  }
}
```


---

## Get Assets Assignments Report

Endpoint: `GET /api/reports/assets/assignments`

Authorization: Required

Description: Get assets with their assignment information

Success Response:


```
{
  "success": true,
  "message": "Assets by employee report generated successfully",
  "data": [
    {
      "asset_id": 1,
      "asset_tag": "ASSET-001",
      "asset_name": "Dell Laptop",
      "employee_id": 5,
      "first_name": "Caleb",
      "middle_name": null,
      "last_name": "Kiprono",
      "assignment_date": "2026-01-01",
      "status": "Assigned"
    }
  ]
```

```
}
```

---

## Get All Expenses Report

Endpoint: `GET /api/reports/expenses/all`

Authorization: Required

Description: Get all expenses without filters

Success Response:

```
{
  "success": true,
  "message": "Assets by employee report generated successfully",
  "data": [
    {
      "expense_id": 1,
      "asset_tag": "ASSET-001",
      "expense_type": "Repair",
      "amount": 250,
      "expense_date": "2026-01-05",
      "vendor": "Tech Repairs Inc"
    }
  ]
}
```

---

## Export Assets Report

Endpoint: `GET /api/reports/assets/export`

Authorization: Required

Description: Export assets report to Excel

Query Parameters: Same as Get Filtered Assets Report

Response: Excel file download

---

### Get Assets By Employee

Endpoint: `GET /api/reports/assets/employee/:employeeId`

Authorization: Required

Description: Get all assets assigned to a specific employee

URL Parameters:

- `employeeId` - Employee ID

---

### Get Assets By Branch

Endpoint: `GET /api/reports/assets/branch/:location`

Authorization: Required

Description: Get all assets at a specific branch

URL Parameters:

- `location` - Branch location or ID

---

### Get Expense Report Data

Endpoint: `GET /api/reports/expenses`

Authorization: Required

Description: Get filtered expenses report with pagination

Query Parameters:

- `asset_id` - Filter by asset ID
- `asset_tag` - Filter by asset tag
- `expense_type_id` - Filter by expense type ID
- `expense_type` - Filter by expense type name
- `from_date` - Start date (YYYY-MM-DD)
- `to_date` - End date (YYYY-MM-DD)
- `min_amount` - Minimum amount
- `max_amount` - Maximum amount

- `vendor` - Filter by vendor name
- `limit` - Number of records per page (default: 20)
- `offset` - Starting record number (default: 0)

Example: `GET /api/reports/expenses?from_date=2025-01-01&to_date=2026-01-08&expense_type=Repair&limit=50&offset=0`

Success Response:


```
{
  "success": true,
  "message": "Expense report data retrieved successfully",
  "data": {
    "expenses": [
      {
        "expense_id": 15,
        "asset_id": 1,
        "asset_tag": "ASSET-001",
        "asset_name": "Dell Laptop",
        "expense_type": "Repair",
        "expense_type_id": 2,
        "amount": "KES 250.00",
        "date": "2026-01-08",
        "description": "Screen replacement",
        "vendor": "Tech Repairs Inc"
      }
    ],
    "totalCount": 145
  }
}
```


---

## Export Expenses Report

Endpoint: `GET /api/reports/expenses/export`

Authorization: Required

Description: Export expenses report to Excel

Query Parameters: Same as Get Expense Report Data (all filters supported)

- `asset_id` - Filter by asset ID
- `asset_tag` - Filter by asset tag
- `expense_type_id` - Filter by expense type ID
- `expense_type` - Filter by expense type name
- `from_date` - Start date (YYYY-MM-DD)
- `to_date` - End date (YYYY-MM-DD)

- `min_amount` - Minimum amount
- `max_amount` - Maximum amount
- `vendor` - Filter by vendor name

Example: `GET /api/reports/expenses/export?from_date=2025-01-01&to_date=2026-01-08`

Response: Excel file download (Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)

Filename: `Expense_Report_YYYY-MM-DD.xlsx`

---

## Get Expenses By Time Period

Endpoint: `GET /api/reports/expenses/time-period`

Authorization: Required (Admin)

Description: Get expenses within a specific time period with statistical analysis

Query Parameters:

- `startDate` - Start date (YYYY-MM-DD) Required
- `endDate` - End date (YYYY-MM-DD) Required

Example: `GET /api/reports/expenses/time-period?startDate=2025-01-01&endDate=2026-01-08`

Success Response:


```
{
  "success": true,
  "message": "Expenses report generated successfully",
  "data": {
    "expenses": [
      {
        "expense_id": 15,
        "asset_tag": "ASSET-001",
        "expense_type": "Repair",
        "amount": 250,
        "expense_date": "2026-01-08",
        "vendor": "Tech Repairs Inc"
      }
    ],
    "summary": {
      "total_expenses": 5420.5,
      "expense_count": 23,
      "average_expense": 235.67
```

```
    }
  }
}
```

---

### Get Assignment Report Data

Endpoint: `GET /api/reports/assignments`

Authorization: Required

Description: Get filtered assignments report with pagination

Query Parameters:

- `asset_tag` - Filter by asset tag (partial match)
- `employee_id` - Filter by employee ID
- `employee_name` - Filter by employee name (partial match)
- `from_date` - Filter assignments from this date (YYYY-MM-DD)
- `to_date` - Filter assignments to this date (YYYY-MM-DD)
- `is_returned` - Filter by return status (true/false)
- `branch_id` - Filter by branch ID
- `department_id` - Filter by department ID
- `limit` - Number of records per page (default: 20)
- `offset` - Starting record number (default: 0)

Example: `GET /api/reports/assignments?is_returned=false&from_date=2025-01-01&limit=20&offset=0`

Success Response:

```
{
  "success": true,
  "message": "Assignments retrieved successfully",
  "data": {
    "assignments": [
      {
        "assignment_id": 10,
        "asset_id": 1,
        "asset_tag": "ASSET-001",
        "asset_name": "Dell Laptop",
        "employee_id": 5,
        "employee_name": "Caleb Kiprono",
        "first_name": "Caleb",
        "middle_name": null,
        "last_name": "1/2026",
        "return_date": "Active",
        "notes": "Laptop for development",
```

```
        "branch": "Head Office",
        "department": "IT"
      }
    ],
    "totalCount": 87
  }
}
```

---

## Export Assignment Report

Endpoint: `GET /api/reports/assignments/export`

Authorization: Required

Description: Export assignments report to Excel

Query Parameters: Same as Get Assignment Report Data (all filters supported)

- `asset_tag` - Filter by asset tag
- `employee_id` - Filter by employee ID
- `employee_name` - Filter by employee name
- `from_date` - Start date (YYYY-MM-DD)
- `to_date` - End date (YYYY-MM-DD)
- `is_returned` - Filter by return status (true/false)
- `branch_id` - Filter by branch ID
- `department_id` - Filter by department ID

Example: `GET /api/reports/assignments/export?is_returned=false&from_date=2025-01-01`

Response: Excel file download (Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)

Filename: `Assignments_Report_YYYY-MM-DD.xlsx`

---

## Get Action Log Report Data

Endpoint: `GET /api/reports/action-logs`

Authorization: Required

Description: Get filtered action logs report with pagination

Query Parameters:

- `action_type` - Filter by action type (e.g., "CREATE", "UPDATE", "DELETE", "ASSIGN", "RETURN")
- `user_id` - Filter by user who performed action
- `entity_type` - Filter by entity type (e.g., "Asset", "Assignment", "Expense", "User")
- `entity_id` - Filter by specific entity ID
- `from_date` - Start date (YYYY-MM-DD)
- `to_date` - End date (YYYY-MM-DD)
- `limit` - Number of records per page (default: 20)
- `offset` - Starting record number (default: 0)

Example: `GET /api/reports/action-logs?action_type=CREATE&entity_type=Asset&limit=50&offset=0`

Success Response:


```
{
  "success": true,
  "message": "Action logs retrieved successfully",
  "data": {
    "logs": [
      {
        "id": 100,
        "action_type": "CREATE",
        "entity_type": "Asset",
        "entity_id": 1,
        "user_id": 5,
        "user_name": "Caleb Kiprono",
        "user_email": "caleb@jiranismart.com",
        "created_at": "2026-01-08T10:30:00Z",
        "description": "Created new asset ASSET-001",
        "details": "{\"asset_tag\":\"ASSET-001\",\"asset_name\":\"Dell Laptop\"}",
        "ip_address": "192.168.1.100"
      }
    ],
    "totalCount": 523
  }
}
```


Note: The `details` field contains JSON string with additional information about the action performed.

Action Types:

- `CREATE` - New record created
- `UPDATE` - Record modified
- `DELETE` - Record deleted

- `ASSIGN` - Asset assigned to employee
- `RETURN` - Asset returned from employee
- `LOGIN` - User login activity
- `LOGOUT` - User logout activity
- `UPLOAD` - File/bulk upload
- `EXPORT` - Report exported

Entity Types:

- `Asset` - Asset records
- `Assignment` - Assignment records
- `Expense` - Expense records
- `User` - User accounts
- `Employee` - Employee records
- `Department` - Department records
- `Branch` - Branch records
- `AssetType` - Asset type records
- `ExpenseType` - Expense type records
- `AssetStatus` - Asset status records

---

## Export Action Log Report

Endpoint: `GET /api/reports/action-logs/export`

Authorization: Required

Description: Export action logs report to Excel

Query Parameters: Same as Get Action Log Report Data (all filters supported)

- `action_type` - Filter by action type
- `user_id` - Filter by user ID
- `entity_type` - Filter by entity type
- `entity_id` - Filter by entity ID
- `from_date` - Start date (YYYY-MM-DD)
- `to_date` - End date (YYYY-MM-DD)

Example: `GET /api/reports/action-logs/export?action_type=CREATE&from_date=2025-01-01&to_date=2026-01-08`

Response: Excel file download (Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)

Filename: `Action_Logs_Report_YYYY-MM-DD.xlsx`

# Bulk Upload

## Bulk Upload Assets

Endpoint: `POST /api/upload/assets`

Authorization: Required (Admin)

Description: Upload multiple assets via Excel file

Content-Type: `multipart/form-data`

Form Data:

- `file` - Excel file (.xlsx) with asset data

Excel File Format:

Required columns:

- `asset_tag` - Unique asset identifier
- `asset_name` - Name of the asset
- `asset_type_id` - Asset type ID
- `purchase_date` - Date (YYYY-MM-DD)
- `purchase_price` - Decimal number
- `status_id` - Status ID
- `branch_id` - Branch ID
- `department_id` - Department ID

Optional columns:

- `description` - Asset description
- `serial_number` - Serial number
- `current_value` - Current value
- `warranty_expiry_date` - Date (YYYY-MM-DD)

Success Response:

```
{
  "success": true,
  "message": "Assets uploaded successfully",
  "data": {
    "total": 50,
    "successful": 48,
    "failed": 2,
    "errors": [
      {
        "row": 5,
        "error": "Duplicate asset tag"
      },
```

```
      {
        "row": 12,
        "error": "Invalid asset type ID"
      }
    ]
  }
}
```

# System Configuration

## Get System Configuration

Endpoint: `GET /api/system-config/`

Authorization: Required (Admin)

Description: Retrieve system configuration settings

Success Response:

```
{
  "success": true,
  "message": "Configuration retrieved successfully",
  "data": {
    "id": 1,
    "company_name": "JSL Systems",
    "company_email": "contact@jslsystems.com",
    "company_phone": "+1-555-0100",
    "company_address": "123 Business Ave",
    "logo_url": "/uploads/logos/company-logo.png",
    "currency": "USD",
    "date_format": "YYYY-MM-DD",
    "timezone": "America/New_York"
  }
}
```

---

## Get Public Configuration

Endpoint: `GET /api/system-config/public`

Authorization: None

Description: Retrieve public system configuration (for login page, etc.)

Success Response:

```
{
  "success": true,
  "message": "Public configuration retrieved successfully",
  "data": {
    "company_name": "JSL Systems",
    "logo_url": "/uploads/logos/company-logo.png"
  }
}
```

---

### Update System Configuration

Endpoint: `PUT /api/system-config/`

Authorization: Required (Admin)

Description: Update system configuration settings

Request Body:

```
{
  "company_name": "JSL Systems Inc.",
  "company_email": "info@jslsystems.com",
  "company_phone": "+1-555-0101",
  "company_address": "456 Corporate Drive",
  "currency": "USD",
  "date_format": "MM/DD/YYYY",
  "timezone": "America/Los_Angeles"
}
```

Success Response:

```
{
  "success": true,
  "message": "Configuration updated successfully",
  "data": {
    "id": 1,
    "company_name": "JSL Systems Inc.",
    ...
  }
}
```

---

### Upload Company Logo

Endpoint: `POST /api/system-config/upload-logo`

Authorization: Required (Admin)

Description: Upload or update company logo

Content-Type: `multipart/form-data`

Form Data:

- `logo` - Image file (max 5MB, image formats only)

Success Response:

```
{
  "success": true,
  "message": "Logo uploaded successfully",
  "data": {
    "logo_url": "/uploads/logos/company-logo-1704715800000.png"
  }
}
```

## Error Responses

All endpoints may return error responses in the following format:

### 400 Bad Request

```
{
  "success": false,
  "message": "Invalid request data",
  "error": "Detailed error message"
}
```

### 401 Unauthorized

```
{
  "success": false,
  "message": "Authentication required",
  "error": "No token provided"
}
```

### 403 Forbidden

```
{
  "success": false,
  "message": "Access denied",
  "error": "Insufficient permissions"
}
```

### 404 Not Found

```
{
  "success": false,
  "message": "Resource not found",
  "error": "Asset with ID 999 not found"
}
```

### 500 Internal Server Error

```
{
  "success": false,
  "message": "Internal server error",
  "error": "Database connection failed"
}
```

# Authentication & Authorization

## Authentication Method

The API uses **JWT (JSON Web Tokens)** for authentication. Tokens are stored in HTTP-only cookies:

- Access Token: Valid for 30 minutes
- Refresh Token: Valid for 7 days

## Cookie Names

- `accessToken` - Used for API authentication
- `refreshToken` - Used for token refresh

## Authorization Roles

The system supports role-based access control:

1. **Admin** - Full system access
2. **Standard User** - Limited access to view and basic operations
3. **Super Admin** - System-level configuration access (specific endpoints)

## Making Authenticated Requests

Authentication is handled automatically via cookies. For external API clients:

1. Login via `/api/auth/login` to receive tokens
2. Tokens are automatically sent with subsequent requests via cookies
3. When access token expires, use `/api/auth/refresh-token` to get a new one
4. Logout via `/api/auth/logout` to clear tokens

# Data Formats

## Date Format

All dates should be in ISO 8601 format: `YYYY-MM-DD`

Example: `2026-01-08`

## DateTime Format

All timestamps are in ISO 8601 format with timezone: `YYYY-MM-DDTHH:mm:ssZ`

Example: `2026-01-08T10:30:00Z`

## Currency

All monetary values are decimal numbers with 2 decimal places.

Example: `1299.99`

## Pagination

Endpoints that support pagination use the following query parameters:

- `limit` - Number of records per page (default: 20, max: 100)
- `offset` - Starting record number (default: 0)

Example:

```
GET /api/reports/assets?limit=50&offset=100
```

Response includes pagination metadata:

```
{
  "success": true,
  "message": "Assets retrieved successfully",
  "data": {
    "items": [...],
    "total": 500,
    "limit": 50,
    "offset": 100,
    "hasMore": true
  }
}
```

## File Upload Limits

- Asset Attachments: 10MB max
- Assignment Attachments: 10MB max
- Expense Attachments: 10MB max
- Company Logo: 5MB max (images only)
- Bulk Upload Excel: Size depends on data volume

# API Testing Examples

## Using cURL

### *Login Example*

```
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "admin@jiranismart.com",
    "password": "password123"
  }' \
  -c cookies.txt
```

### *Get Assets with Authentication*

```
curl -X GET http://localhost:3000/api/assets/ \
  -H "Content-Type: application/json" \
  -b cookies.txt
```

### *Create Asset*

```
curl -X POST http://localhost:3000/api/assets/ \
  -H "Content-Type: application/json" \
  -b cookies.txt \
  -d '{
    "asset_tag": "ASSET-100",
    "asset_name": "MacBook Pro",
    "asset_type_id": 1,
    "purchase_date": "2026-01-08",
    "purchase_price": 2500.00,
    "status_id": 1,
    "branch_id": 1,
    "department_id": 1
  }'
```

### *Upload Asset Attachment*

```
curl -X POST http://localhost:3000/api/asset-attachments/1/attachments \
  -H "Content-Type: multipart/form-data" \
  -b cookies.txt \
  -F "file=@/path/to/invoice.pdf"
```

### *Search Assets*

```
curl -X GET "http://localhost:3000/api/assets/search?asset_name=laptop&status_id=1"
\
  -H "Content-Type: application/json" \
  -b cookies.txt
```

*Export Report*

```
curl -X GET "http://localhost:3000/api/reports/assets/export?status_id=1" \
  -b cookies.txt \
  --output Asset_Report.xlsx
```

## Using Postman

1. **Set Base URL:** Create an environment variable `base_url` = `http://localhost:3000/api`
2. **Login:** POST to `{{base_url}}/auth/login` - Cookies will be saved automatically
3. **Make Requests:** Use `{{base_url}}/assets/` for subsequent requests
4. **File Upload:** Use form-data with file field for attachment endpoints

## Using JavaScript (Fetch API)

```
// Login
const login = async () => {
  const response = await fetch('http://localhost:3000/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({
      email: 'admin@jiranismart.com',
      password: 'password123'
    })
  });
  return await response.json();
};

// Get Assets
const getAssets = async () => {
  const response = await fetch('http://localhost:3000/api/assets/', {
    method: 'GET',
    credentials: 'include'
  });
  return await response.json();
};

// Create Asset
const createAsset = async (assetData) => {
  const response = await fetch('http://localhost:3000/api/assets/', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify(assetData)
  });
  return await response.json();
};

// Upload File
```

```
const uploadFile = async (assetId, file) => {
  const formData = new FormData();
  formData.append('file', file);

  const response = await fetch(`http://localhost:3000/api/asset-
attachments/${assetId}/attachments`, {
    method: 'POST',
    credentials: 'include',
    body: formData
  });
  return await response.json();
};
```

# Common Integration Patterns

## Workflow 1: Complete Asset Lifecycle

```
1. POST /api/auth/login
   → Login and receive cookies

2. POST /api/assets/
   → Create new asset

3. POST /api/asset-attachments/:assetId/attachments
   → Upload invoice/receipt

4. POST /api/assignments/
   → Assign asset to employee

5. POST /api/assignment-attachments/:assignmentId/attachments
   → Upload handover documents

6. GET /api/reports/assignments
   → Monitor active assignments

7. PUT /api/assignments/:id/return
   → Return asset when done

8. POST /api/expenses/
   → Record any maintenance expenses

9. GET /api/reports/assets/:id
   → View complete asset history
```

## Workflow 2: Monthly Reporting

```
1. POST /api/auth/login
   → Authenticate

2. GET /api/reports/assets?limit=100&offset=0
   → Get assets report data (paginated)

3. GET /api/reports/expenses?from_date=2026-01-01&to_date=2026-01-31
   → Get monthly expenses

4. GET /api/reports/assignments?from_date=2026-01-01
   → Get monthly assignments

5. GET /api/reports/assets/export
   → Export full report to Excel

6. GET /api/reports/expenses/export?from_date=2026-01-01&to_date=2026-01-31
   → Export expenses to Excel
```

```
7. GET /api/reports/action-logs?from_date=2026-01-01
   → Audit trail for the month
```

## Workflow 3: Bulk Operations

```
1. POST /api/auth/login
   → Authenticate as Admin

2. POST /api/upload/assets
   → Bulk upload assets from Excel

3. GET /api/assets/
   → Verify uploaded assets

4. GET /api/reports/assets?limit=1000
   → Review all assets

5. POST /api/assets/:id (for each correction needed)
   → Fix any data issues
```

## Rate Limiting

Currently, there are no explicit rate limits implemented. However, it is recommended to:

- Avoid excessive concurrent requests
- Implement client-side throttling
- Use pagination for large datasets

# Troubleshooting

## Common Issues

### *1. Authentication Failures*

Problem: 401 Unauthorized

```
{
  "success": false,
  "message": "Authentication required"
}
```

Solutions:

- Ensure you've logged in via `/api/auth/login`
- Check that cookies are being sent with requests (`credentials: 'include'` in fetch)
- Verify access token hasn't expired (30 min lifetime)
- Try refreshing token via `/api/auth/refresh-token`

---

### *2. File Upload Failures*

Problem: File too large or invalid format

```
{
  "success": false,
  "message": "File size exceeds maximum limit of 10MB"
}
```

Solutions:

- Compress large files before upload
- Verify file format is supported
- Check Content-Type is `multipart/form-data`
- Ensure field name is correct (`file` for attachments, `logo` for company logo)

---

### *3. Pagination Issues*

Problem: Missing or incorrect pagination

```
{
  "success": false,
  "message": "Invalid pagination parameters"
}
```

Solutions:

- Ensure `limit` is a positive integer
- Ensure `offset` is 0 or positive
- Don't exceed `limit` of 100 per request
- Use correct query string format: `?limit=20&offset=0`

---

### *4. Foreign Key Violations*

Problem: Referenced entity doesn't exist

```
{
  "success": false,
  "message": "Invalid asset_type_id: Asset type with ID 99 not found"
}
```

Solutions:

- Verify the referenced entity exists (e.g., asset type, department, branch)
- Get valid IDs from lookup endpoints first
- Check for typos in ID values

---

### *5. Duplicate Entries*

Problem: Unique constraint violation

```
{
```

```
  "success": false,
  "message": "Asset tag 'ASSET-001' already exists"
}
```

Solutions:

- Use unique asset tags
- Check existing records before creating new ones
- Use search endpoint to verify uniqueness

---

### *6. Permission Denied*

Problem: 403 Forbidden

```
{
  "success": false,
  "message": "Access denied. Admin role required."
}
```

Solutions:

- Verify user has correct role for the operation
- Check authorization requirements in documentation
- Contact administrator to update user role if needed

---

## Debug Mode

For development, you can enable detailed error logging:

5. Set environment variable: `NODE_ENV=development`
6. Check server logs for detailed stack traces
7. Use browser DevTools Network tab to inspect requests/responses

---

## Getting Help

If you encounter issues not covered here:

8. Check the server logs for detailed error messages
9. Verify all required fields are included in request body
10. Test with a simple cURL command first
11. Contact support at: fmbugua@jiranismart.com

# Changelog

## Version 2.0 (January 8, 2026)

- Complete API documentation update with all endpoints
- Added comprehensive Quick Reference table with all 90+ endpoints
- Added System Configuration endpoints
- Added Action Log reporting with entity and action type details
- Enhanced filtering and pagination across all report endpoints
- Added attachment support for assets, assignments, and expenses
- Improved authentication with refresh token mechanism
- Added bulk upload functionality for assets
- Added detailed request/response examples for every endpoint
- Added cURL, JavaScript, and Postman usage examples
- Added common integration workflow patterns
- Added comprehensive troubleshooting guide
- Added error handling examples for each endpoint
- Documented all query parameters and filters
- Added file upload specifications and limits

## Version 1.0 (September 20, 2025)

- Initial API release
- Basic CRUD operations for assets, assignments, and expenses
- User authentication and authorization
- Basic reporting functionality

## Glossary

Asset - Physical or digital resource owned by the organization (laptops, furniture, software licenses, etc.)

Asset Tag - Unique identifier assigned to each asset for tracking purposes

Assignment - Record of an asset being allocated to an employee with assignment and return dates

Attachment - File (document, image, receipt) associated with an asset, assignment, or expense

Authentication - Process of verifying user identity via login credentials

Authorization - Process of determining if authenticated user has permission for requested action

Branch - Physical location or office of the organization

Bulk Upload - Process of importing multiple records at once via Excel file

Department - Organizational unit grouping employees by function (IT, HR, Finance, etc.)

Expense - Cost incurred for maintaining, repairing, or upgrading an asset

Expense Type - Category of expense (Repair, Maintenance, Upgrade, etc.)

JWT (JSON Web Token) - Secure token used for authentication, stored in HTTP-only cookies

Pagination - Technique to retrieve large datasets in smaller chunks using limit/offset parameters

Refresh Token - Long-lived token used to obtain new access tokens without re-login

Role - User permission level (Admin, Standard User, Super Admin)

Status - Current state of an asset (Available, In Use, Under Repair, Retired, etc.)

# API Limits & Constraints

## Request Limits

- Maximum Upload Size: 10MB for attachments, 5MB for company logo
- Pagination Limit: Maximum 100 records per request
- Query String Length: 2048 characters maximum
- Request Timeout: 30 seconds for standard requests, 120 seconds for export operations

## Data Constraints

- Asset Tag: 50 characters maximum, must be unique
- Email: 255 characters maximum, must be valid format
- Password: Minimum 8 characters, must contain letters and numbers
- Phone Number: 20 characters maximum
- Description Fields: 1000 characters maximum
- File Names: 255 characters maximum

## Rate Limits

- Currently no explicit rate limits implemented
- Recommended: Maximum 100 requests per minute per user
- Bulk operations should be performed during off-peak hours

# Security Best Practices

## For API Consumers

1. **Protect Credentials**
- Never commit credentials to version control
- Use environment variables for sensitive data
- Rotate passwords regularly

2. **Token Management**
- Store tokens securely (HTTP-only cookies recommended)
- Implement token refresh logic
- Clear tokens on logout

3. **Input Validation**
- Validate all input on client side before sending
- Sanitize user input to prevent injection attacks
- Use parameterized queries

4. **HTTPS**
- Always use HTTPS in production
- Verify SSL certificates
- Don't expose API over HTTP

5. **Error Handling**
- Don't expose sensitive information in errors
- Log errors for debugging
- Provide user-friendly error messages

## For API Administrators

1. **Access Control**
- Follow principle of least privilege
- Regularly review user permissions
- Disable inactive accounts

2. **Monitoring**
- Log all API requests
- Monitor for unusual activity
- Set up alerts for failed authentication attempts

3. **Backups**
- Regular database backups
- Test restore procedures
- Keep backups encrypted and off-site

4. **Updates**
- Keep dependencies updated
- Apply security patches promptly
- Review security advisories

## Support

For API support or questions, please contact:

Technical Support:

- Email: fmbugua@jiranismart.com
- Phone: +254 712 345 678
- Hours: Monday - Friday, 8:00 AM - 5:00 PM EAT

Documentation:

- API Docs: This file (API_DOCUMENTATION.md)
- GitHub: (If applicable - add repository link)

Environment URLs:

- Development: http://localhost:3000/api
- Staging: (Add staging URL if applicable)
- Production: (Add production URL if applicable)

Additional Resources:

- Database Schema Documentation
- System Architecture Diagram
- Deployment Guide
- User Manual

---

Last Updated: January 8, 2026

Document Version: 2.0

API Version: 2.0

---

Note: This documentation is based on the current implementation as of January 8, 2026. Always refer to the latest version of this document for accurate API information. For the most up-to-date endpoint specifications, consult the source code in the `/src/routes` directory.